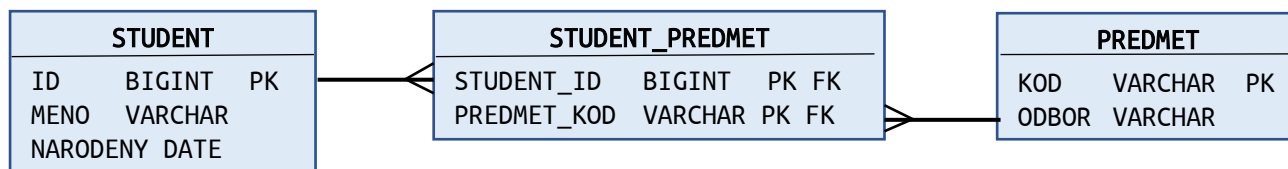


Implementačná úloha 1.

Na obrázku je zobrazený diagram databázy s údajmi o študentoch a predmetoch, ktoré majú zapísané.



Adresár **dbapp** obsahuje maven project pre vývoj aplikácie pracujúcej s touto databázou.

Projekt obsahuje balík **skola** s tromi zdrojovými súbormi:

- **Student.java**: Dátová trieda s údajmi o študentoch. Obsahuje zatiaľ len deklarácie dátových členov (a tiež metódy hashCode, equals, a toString)
- **Predmet.java**: Dátová trieda s údajmi o predmetoch. Obsahuje zatiaľ len deklarácie dátových členov (a tiež metódy hashCode, equals, a toString)
- **DbApp.java**: Obsahuje deklarácie troch metód pre prácu s uvedenou databázou.
 - **int pocetPredmetov(EntityManager em, String meno)**
zistí počet predmetov, ktoré má študent s daným meno zapísané.
 - **Predmet novyPredmet(EntityManager em, String kodPredmetu)**
vytvorí nový predmet.
 - **void zapisatStudenta(EntityManager em, String meno, String kodPredmetu)**
zapiše študenta na predmet

Vašou úlohou je implementácia týchto troch **metód podľa špecifikácie**.

Presnú **špecifikáciu nájdete v zdrojovom súbore v komentári** k deklaráciám týchto metód.

Pokyny k implementácii:

Aby ste mohli metódy pracujúce s databázou implementovať, je nutné najprv dokončiť implementáciu tried **Student** a **Predmet**. Existujúci kód v súboroch nie je potrebné modifikovať, názvy a typy dátových členov nemeniť! Stačí ho doplniť o:

- **gettre a settre** pre dátové členy
- potrebné **JPA anotácie**, tak aby sa entitné triedy mapovali na databázové tabuľky s presne takou štruktúrou, ako je uvedená v diagrame hore a pritom:
 - kľúč ID tabuľky STUDENT bude **autogenerovaný**
 - kľúč KOD tabuľky PREDMET nie je autogenerovaný.

Odporúčam postupovať tak, že najprv dokončíte implementáciu entitných tried a z nich si necháte vygenerovať tabuľky.

Dôležité je, aby vygenerované tabuľky mali **presne tú istú štruktúru**, ako je uvedená v diagrame hore. Názvy tabuliek, stĺpcov a ich typy musia byť zachované (BIGINT odpovedá typu long). Skontrolujte si predovšetkým štruktúru prepojovacej tabuľky.

*Okrem troch metód, ktoré máte implementovať, obsahuje súbor DbApp.java aj **main** funkciu. Táto funkcia slúži len pre vaše testovanie, nehodnotí sa. Je už implementovaná a používa lokálnu derby/sample databázu. Súbor **persistence.xml** s konfiguráciou pripojenia na túto databázu už máte v projekte tiež pripravený, môžete si ho však upraviť, ak chcete pre otestovanie použiť inú databázu. Takisto implementáciu funkcie main si môžete prispôsobiť podľa potreby.*

Pokyny pre odovzdanie:

Balík **skola** so všetkými súbormi, ktoré sú v ňom, zozipujte. Zip súbor nazvite **skola.zip** a odovzdajte do miesta odovzdania v AISe.

Žiadam vás, na zipovanie použite len formát **zip** (nie rar či iné archívne formáty). Zipujte len samotný adresár **skola**, nie celý projekt! Je to dôležité pre včasné vyhodnotenie vašich riešení.

Implementačná úloha 2.

V adresári **restapp** nájdete pripravený maven projekt pre vývoj webovej aplikácie s REST servisom.

Projekt obsahuje:

- súbor **pom.xml** so všetkými závislosťami potrebnými pre vývoj servisu. Nie je ho potrebné dopĺňať.
- balík **rest**. **V tomto balíku budete vytvárať všetky triedy** (triedu implementujúcu servis aj dátovú/é triedu)

V balíku rest sa už nachádza konfiguračná trieda REST aplikácie JAXRSConfiguration. Netreba ju meniť.

Špecifikácia servisu

Servis bude sprístupňovať informácie o poistných zmluvách a v nich poistených osobách.

Informácie o poistnej zmluve predstavujú resource, ktorého stav reprezentovaný vo formáte XML má štruktúru:

```
<?xml version="1.0" encoding="UTF-8"?>
<zmluva>
  <id>Z123</id>
  <majitel>Mrkvicka</majitel >
  <poistenec>Fenko Mrkvicka</poistenec>
  <poistenec>Janko Hrasko</poistenec>
</zmluva>
```

kde:

- element **id** obsahuje jednoznačný identifikátor zmluvy. Teda musí byť zadaný.
- element **majitel** obsahuje meno majiteľa zmluvy.
- elementy **poistenec** obsahujú mená poistených osôb.

Poznámka. Uvedený dokument je ukážka. XML dokumenty, s ktorými servis pracuje budú mať elementy s rovnakými názvami a štruktúrou, odlišovať sa budú len textovým obsahom a počtom elementov. (poistencov môže byť viac ale nemusí byť žiadny. Poradie elementov nie je dôležité.)

Informácie o zmluvách (stav resoursov) budú udržiavané len v pamäti servisu. Vytvorte si pre to vhodné dátové a použite anotáciu **@javax.inject.Singleton**. Nezabudnite tiež na anotáciu **@XmlRootElement**

Servis inicializujte tak, že po štarte (deployment) bude už vytvorená jedna zmluva, ktorá má id **Z123**, majiteľ je **“Mrkvicka”** a nemá **žiadnych poistencov**.

Ďalšie poistné zmluvy bude možné pridávať volaním metódy POST pre koreňový resource servisu.

Koreňový resource nazvite **poistenie** a implementujete pre neho metódu:

- **POST** pre MediaType.APPLICATION_XML: Slúži pre vytvorenie nového resourcu s informáciami o poistnej zmluve. Akceptuje XML dokument podľa horeuvedenej špecifikácie. Preverí, či zmluva z daným identifikátorom už neexistuje. Ak neexistuje, vytvorí nový resource, inak neurobí nič. Metóda nič nevracia.

Subresource s URI **poistenie/{id}**, kde reťazec **id** je identifikátor zmluvy, sprístupňuje konkrétnu poistnú zmluvu. Pre tento subresource implementujte nasledujúce metódy:

- **GET** pre MediaType.APPLICATION_XML: Vyhľadá zmluvu podľa identifikátora zadaného v URL a vráti informácie ako XML dokument s horeuvedenou štruktúrou.
Ak zmluva neexistuje, malo by sa vrátiť HTTP 204 alebo 404 (*Pomôcka, metóda vráti jednoducho null*)
- **GET** pre MediaType.TEXT_PLAIN: Vyhľadá zmluvu a vráti počet poistených osôb.
Ak zmluva neexistuje, vráti text **“neplatna zmluva”**.
- **POST** pre MediaType.TEXT_PLAIN : Slúži na pridanie ďalšej poistenej osoby do zmluvy, očakáva reťazec s menom osoby a vracia takisto textový reťazec (MediaType.TEXT_PLAIN).
Vyhľadá zmluvu podľa id a preverí, či osoba s daným menom sa už nie je medzi poistenými osobami.
Ak nie je, pridá ju do zoznamu poistených osôb a vráti **poradové číslo osoby** v zozname, pričom číslovanie začína od 0.
Ak osoba je už medzi poistenými osobami, vráti jej poradové číslo, ale do zoznamu ju znovu nepridáva.
Ak zmluva s daným id neexistuje, vráti metóda text **“neplatna zmluva”**.

Jednotlivé poistené osoby v zmluve sú prístupné ako subresoursy s URI **poistenie/{id}/{no}**, kde reťazec **id** je identifikátor zmluvy a **no** je poradové číslo v zozname poistených osôb (číslovanie začína od 0).

Pre tento subresource implementujte nasledujúce metódy:

- **GET** pre MediaType.TEXT_PLAIN: Vráti meno poistenej osoby s daný poradovým číslom.
Ak zmluva s daným id neexistuje, vráti metóda text “**neplatna zmluva**”.
Ak osoba s daným id neexistuje, vráti metóda text “**neplatna osoba**”.
- **DELETE** Odstráni položku zo zoznamu poistených osôb.
Metóda neočakáva ani nevracia žiadne dáta.
Poznámka: Po odstránení osoby zo zoznamu poradové čísla ostatných poistených osôb v zozname môžu ale nemusia zmeniť. Nebude sa to overovať unit-testami.

Pokyny pre odovzdanie:

Balík **rest** so všetkými súbormi zozipujte. Zip súbor nazvite **rest.zip** a odovzdajte do miesta odovzdania v **AISe**.

Žiadam vás, na zipovanie použite len formát **zip** (*nie rar či iné archívne formáty*). Zipujte len samotný adresár **rest**, nie celý projekt! Je to dôležité pre včasné vyhodnotenie vašich riešení.